

React JS 常用表單元件

建立了一個帶有文字輸入框的表單組件：

`import { useState } from 'react';`：這是從 React 函式庫中匯入 `useState` Hook 函式。`useState` 用於在函式組件中添加狀態管理能力。

`export default function FormText() { ... }`：這是一個預設匯出的函式組件，名稱為 `FormText`。

`const [inputValue, setInputValue] = useState('');`：這行程式碼使用 `useState` 鉤子函式來宣告一個名為 `inputValue` 的狀態變數，並將其初始值設定為空字串。`setInputValue` 是用於更新 `inputValue` 狀態變數的函式。

`const handleChange = (event) => { ... }`：這是一個名為 `handleChange` 的事件處理函式，用於處理文字輸入框的變化事件。當輸入框的值發生變化時，該函式將更新 `inputValue` 的狀態。

`<input type="text" value={inputValue} onChange={handleChange} />`：這是一個文字輸入框，其值被綁定到 `inputValue` 狀態變數，即目前輸入框的值。當輸入框的值發生變化時，`onChange` 事件將觸發 `handleChange` 函式。

`<p>Input Value: {inputValue}</p>`：這是一個顯示目前輸入值的段落。它使用插值將 `inputValue` 的值顯示在頁面上。

整個組件的運作方式如下：

初始化一個名為 `inputValue` 的狀態變數，並將其初始值設定為空字串。

渲染一個帶有文字輸入框的表單，並將輸入框的值綁定到 `inputValue`。

當使用者在輸入框中鍵入時，`onChange` 事件會呼叫 `handleChange` 函式，該函式會更新 `inputValue` 的值。

更新後的 `inputValue` 的值會重新渲染到頁面上的段落中，以顯示目前輸入的值。

```
import { useState } from 'react';
```

```
export default function FormText() {  
  const [inputValue, setInputValue] = useState('');
```

```
const handleChange = (event) => {
  setInputValue(event.target.value);
};

return (
  <div>
    <form>
      <label>Input Value:
        <input type="text" value={inputValue}
onChange={handleChange} />
      </label>
      <p>Input Value: {inputValue}</p>
    </form>
  </div>
)
};
```

建立了一個帶有複選框的表單組件：

`import { useState } from "react";`：這是從 `React` 函式庫中匯入 `useState` 鉤子函式。`useState` 用於在函式組件中添加狀態管理能力。

`function FormCheckbox() { ... }`：這是一個沒有預設匯出的函式，名稱為 `FormCheckbox`。

`const [isChecked, setIsChecked] = useState(false);`：這行程式碼使用 `useState` 鉤子函式來宣告一個名為 `isChecked` 的狀態變數，並將其初始值設定為 `false`。`setIsChecked` 是用於更新 `isChecked` 狀態變數的函式。

`const handleChange = (event) => { ... }`：這是一個名為 `handleChange` 的事件處理函式，用於處理複選框的變化事件。當複選框的狀態變化時，該函式將更新 `isChecked` 的狀態。

`<input type="checkbox" name="color" checked={isChecked} onChange={handleChange}/>`：這是一個複選框，其勾選狀態被綁定到 `isChecked` 狀態變數，即目前複選框的狀態。當複選框的勾選狀態變化時，`onChange` 事件會觸發 `handleChange` 函式。

`{isChecked && <div>Blue is selected!</div>}`：這是一個條件渲染的語句。如果 `isChecked` 為真，則顯示一個顯示 "Blue is selected!" 的 `<div>` 元素。

`{!isChecked && <div>Blue is Deselected!</div>}`：這也是一個條件渲染的語句。如果 `isChecked` 為假，則顯示一個顯示 "Blue is Deselected!" 的 `<div>` 元素。

整個組件的運作方式如下：

初始化一個名為 `isChecked` 的狀態變數，並將其初始值設定為 `false`，表示複選框未被勾選。

渲染一個帶有複選框的表單，並將複選框的勾選狀態綁定到 `isChecked`。

當使用者勾選或取消勾選複選框時，`onChange` 事件會呼叫 `handleChange` 函式，該函式會更新 `isChecked` 的值。

根據 `isChecked` 的值，顯示不同的 `<div>` 元素，以顯示相應的訊息。如果 `isChecked` 為真，顯示 "Blue is selected!"；如果 `isChecked` 為假，顯示 "Blue is Deselected!"。

```

import { useState } from "react";

function FormCheckbox() {
  const [isChecked, setIsChecked] = useState(false);

  const handleChange = (event) => {
    setIsChecked(event.target.checked);
  };

  return (
    <form>
      <label htmlFor="color">
        <input type="checkbox" name="color" checked={isChecked}
onChange={handleChange}/>
        Blue
      </label>

      {isChecked && <div>Blue is selected!</div>}
      {!isChecked && <div>Blue is Deselected!</div>}
    </form>
  );
}

export default FormCheckbox;

```

React 的函式組件，它建立了一個包含下拉選單的表單組件。

`import { useState } from "react";`：這是從 React 函式庫中匯入 `useState` 鉤子函式。`useState` 用於在函式組件中添加狀態管理能力。

`export default function FormDropdown() { ... }`：這是一個預設匯出的函式組件，名稱為 `FormDropdown`。

`const [selectedOption, setSelectedOption] = useState("option1");`：這行程式碼使用 `useState` 鉤子函式來宣告一個名為 `selectedOption` 的狀態變數，並將其初始值設定為 `"option1"`。`setSelectedOption` 是用於更新 `selectedOption` 狀態變數的函式。

`const handleDropdownChange = (event) => { ... }`：這是一個名為 `handleDropdownChange` 的事件處理函式，用於處理下拉選單的變化事件。當下拉選單的值變化時，該函式會更新 `selectedOption` 的狀態。

`<select value={selectedOption} onChange={handleDropdownChange}> ... </select>`：這是一個下拉選單，其值被綁定到 `selectedOption` 狀態變數，即目前選擇的選項。當下拉選單的值變化時，`onChange` 事件會觸發 `handleDropdownChange` 函式。

`<p>Selected option: {selectedOption}</p>`：這是一個顯示目前選擇選項的段落。它使用插值將 `selectedOption` 的值顯示在頁面上。

整個組件的運作方式如下：

初始化一個名為 `selectedOption` 的狀態變數，並將其初始值設定為 `"option1"`，表示預設選擇第一個選項。

渲染一個包含下拉選單的表單，並將下拉選單的值綁定到 `selectedOption`。

當使用者在下拉選單中選擇其他選項時，`onChange` 事件會呼叫 `handleDropdownChange` 函式，該函式會更新 `selectedOption` 的值。

更新後的 `selectedOption` 的值會重新渲染到頁面上的段落中，以顯示目前選擇的選項。

```
import { useState } from "react";
export default function FormDropdown() {
  const [selectedOption, setSelectedOption] = useState("option1");

  const handleDropdownChange = (event) => {
    setSelectedOption(event.target.value);
  };

  return (
    <div>
      <label>
        Select an option:
        <select value={selectedOption}
          onChange={handleDropdownChange}>
          <option value="option1">Option 1</option>
```

```

        <option value="option2">Option 2</option>
        <option value="option3">Option 3</option>
    </select>
</label>
<p>Selected option: {selectedOption}</p>
</div>
);
}

```

React 的函式組件，它建立了一個包含多個表單輸入欄位的表單組件：

`import { useState } from "react";`：這是從 React 函式庫中匯入 `useState` 鉤子函式。`useState` 用於在函式組件中添加狀態管理能力。

`export default function FormMultiple() { ... }`：這是一個預設匯出的函式組件，名稱為 `FormMultiple`。

`const [formData, setFormData] = useState({name: "", email: "", message: ""});`：這行程式碼使用 `useState` 鉤子函式來宣告一個名為 `formData` 的狀態變數，並將其初始值設定為一個物件，該物件包含 `name`、`email` 和 `message` 三個屬性，其值都為空字串。`setFormData` 是用於更新 `formData` 狀態變數的函式。

`const handleChange = (event) => { ... }`：這是一個名為 `handleChange` 的事件處理函式，用於處理表單輸入欄位的變化事件。當表單輸入欄位的值變化時，該函式會更新 `formData` 的狀態。它使用解構賦值將事件目標的 `name` 和 `value` 屬性取出，然後使用展開運算子將原有的 `formData` 物件複製一份，並將對應的欄位值更新為新的值。

`const handleSubmit = (event) => { ... }`：這是一個名為 `handleSubmit` 的事件處理函式，用於處理表單提交事件。它在表單提交時阻止預設行為（避免頁面刷新），然後使用 `alert` 函式彈出包含表單數據的訊息。

`<form onSubmit={handleSubmit}> ... </form>`：這是一個包含 `onSubmit` 事件處理器的表單元素，當使用者提交表單時，會觸發 `handleSubmit` 函式。

`<input type="text" id="name" name="name" value={formData.name} onChange={handleChange}/>`：這是一個文字輸入欄位，它的值被綁定到

`formData.name`，並在值變化時觸發 `handleChange` 函式。

`<input type="email" id="email" name="email" value={formData.email} onChange={handleChange}/>`：這是一個電子郵件輸入欄位，它的值被綁定到 `formData.email`，並在值變化時觸發 `handleChange` 函式。

`<textarea id="message" name="message" value={formData.message} onChange={handleChange}/>`：這是一個多行文字輸入欄位（文本區域），它的值被綁定到 `formData.message`，並在值變化時觸發 `handleChange` 函式。

`<button type="submit">Submit</button>`：這是一個提交按鈕，用於提交表單。當使用者點擊該按鈕時，將觸發表單的提交事件。

整個組件的運作方式如下：

初始化一個名為 `formData` 的狀態變數，並將其初始值設定為一個包含 `name`、`email` 和 `message` 三個屬性的物件，其值都為空字串。

渲染一個包含多個表單輸入欄位的表單。

每個輸入欄位都被綁定到對應的 `formData` 屬性，並在值變化時觸發 `handleChange` 函式進行更新。

當使用者提交表單時，`onSubmit` 事件會觸發 `handleSubmit` 函式，彈出一個 `alert` 訊息顯示表單的數據。

```
import { useState } from "react";
```

```
export default function FormMultiple() {
```

```
  const [formData, setFormData] = useState({name: "",email: "",message: ""});
```

```
  const handleChange = (event) => {
```

```
    const { name, value } = event.target;
```

```
    setFormData((prevFormData) => ({ ...prevFormData, [name]: value }));
```

```
  };
```

```
  const handleSubmit = (event) => {
```

```
    event.preventDefault();
```

```
    alert(`Name: ${formData.name}, Email: ${formData.email}, Message: ${formData.message}`
```

```

    );
};

return (
  <form onSubmit={handleSubmit}>
    <label htmlFor="name">Name:</label>
    <input type="text" id="name" name="name" value={formData.name}
onChange={handleChange}/>
    <p/>
    <label htmlFor="email">Email:</label>
    <input type="email" id="email" name="email" value={formData.email}
onChange={handleChange}/>
    <p/>
    <label htmlFor="message">Message:</label>
    <textarea id="message" name="message" value={formData.message}
onChange={handleChange}/>
    <p/>
    <button type="submit">Submit</button>
  </form>
);
}

```

這是一個使用 **React** 的函式組件，它建立了一個帶有驗證功能的表單組件：

`import { useState } from 'react';`：這是從 **React** 函式庫中匯入 **useState** 鉤子函式。**useState** 用於在函式組件中增加狀態管理能力。

`function FormValidate() { ... }`：這是一個名為 **FormValidate** 的函式組件。

`const [inputValue, setInputValue] = useState('');`：這行程式碼使用 **useState** 鉤子函式來宣告一個名為 **inputValue** 的狀態變數，並將其初始值設定為空字串。**setInputValue** 是用於更新 **inputValue** 狀態變數的函式。

`const [inputError, setInputError] = useState(null);`：這行程式碼使用 **useState** 鉤子函式來宣告一個名為 **inputError** 的狀態變數，並將其初始值設定為 **null**。**setInputError** 是用於更新 **inputError** 狀態變數的函式。

`function handleChange(event) { ... }`：這是一個名為 `handleChange` 的事件處理函式，用於處理輸入欄位值的變化事件。它從事件目標中取得新的值，並使用 `setInputValue` 更新 `inputValue` 狀態變數。同時，它檢查新值的長度，如果小於 5，則使用 `setInputError` 設定錯誤訊息；否則，將 `inputError` 設為 `null`。

`function handleSubmit(event) { ... }`：這是一個名為 `handleSubmit` 的事件處理函式，用於處理表單提交事件。它阻止預設的表單提交行為，並檢查 `inputValue` 的長度。如果 `inputValue` 的長度大於等於 5，則可以提交表單；否則，使用 `setInputError` 設定錯誤訊息。

`<form onSubmit={handleSubmit}> ... </form>`：這是一個包含 `onSubmit` 事件處理器的表單元素，當使用者提交表單時，會觸發 `handleSubmit` 函式。

`<input type="text" value={inputValue} onChange={handleChange} />`：這是一個文字輸入欄位，它的值被綁定到 `inputValue`，並在值變化時觸發 `handleChange` 函式。

`{inputError && <div style={{ color: 'red' }}>{inputError}</div>}`：這是一個條件渲染的區塊，當 `inputError` 存在時，會渲染一個紅色文字的 `div` 元素，顯示錯誤訊息。

`<button type="submit">Submit</button>`：這是一個提交按鈕，用於提交表單。當使用者點擊該按鈕時，將觸發表單的提交事件。

整個組件的運作方式如下：

初始化一個名為 `inputValue` 的狀態變數，並將其初始值設定為空字串。

初始化一個名為 `inputError` 的狀態變數，並將其初始值設定為 `null`。

渲染一個包含一個文字輸入欄位和一個提交按鈕的表單。

輸入欄位的值被綁定到 `inputValue` 狀態變數，並在值變化時觸發 `handleChange` 函式進行更新。

如果輸入欄位的值長度小於 5，則會顯示一個紅色的錯誤訊息。

當使用者提交表單時，如果輸入欄位的值長度大於等於 5，則可以提交表單；否則，會顯示一個紅色的錯誤訊息。

```

import { useState } from 'react';

function FormValidate() {
  const [inputValue, setInputValue] = useState("");
  const [inputError, setInputError] = useState(null);

  function handleInputChange(event) {
    const value = event.target.value;
    setInputValue(value);

    if (value.length < 5) {
      setInputError('Input must be at least 5 characters');
    } else {
      setInputError(null);
    }
  }

  function handleSubmit(event) {
    event.preventDefault();
    if (inputValue.length >= 5) {
      // submit form
    } else {
      setInputError('Input must be at least 5 characters');
    }
  }

  return (
    <form onSubmit={handleSubmit}>
      <label>
        Fruit:
        <input type="text" value={inputValue} onChange={handleInputChange} />
      </label>
      {inputError && <div style={{ color: 'red' }}>{inputError}</div>}
      <button type="submit">Submit</button>
    </form>
  );
}

export default FormValidate;

```

這是一個使用 React 的函式組件，建立了一個無控制組件（Uncontrolled Component）的表單。讓我逐步解釋程式碼的功能：

`import { useRef } from "react";`：這是從 React 函式庫中匯入 `useRef` 鉤子函式。
`useRef` 用於在函式組件中建立一個可讓你存取 DOM 元素的參照。

`export default function UncontrollerForm() { ... }`：這是一個匯出的函式組件，名為 `UncontrollerForm`。

`const selectRef = useRef(null);`：這行程式碼使用 `useRef` 鉤子函式來建立一個名為 `selectRef` 的參照。這個參照將用於存取 `select` 元素。

`const checkboxRef = useRef(null);`：這行程式碼使用 `useRef` 鉤子函式來建立一個名為 `checkboxRef` 的參照。這個參照將用於存取 `checkbox` 元素。

`const inputRef = useRef(null);`：這行程式碼使用 `useRef` 鉤子函式來建立一個名為 `inputRef` 的參照。這個參照將用於存取 `input` 元素。

`function handleSubmit(event) { ... }`：這是一個名為 `handleSubmit` 的事件處理函式，用於處理表單提交事件。它阻止預設的表單提交行為，並從 `inputRef` 參照中取得 `input` 元素的值，從 `selectRef` 參照中取得 `select` 元素的值，並從 `checkboxRef` 參照中取得 `checkbox` 元素的選中狀態，並將這些值輸出到控制台。

`<form onSubmit={handleSubmit}> ... </form>`：這是一個包含 `onSubmit` 事件處理器的表單元素，當使用者提交表單時，會觸發 `handleSubmit` 函式。

`<input ref={inputRef} type="text" />`：這是一個文字輸入欄位，使用 `ref` 屬性將 `inputRef` 參照指定給該元素，以便在需要時能夠存取 `input` 元素。

`<select ref={selectRef}> ... </select>`：這是一個下拉選單，使用 `ref` 屬性將 `selectRef` 參照指定給該元素，以便在需要時能夠存取 `select` 元素。

`<input type="checkbox" ref={checkboxRef} />`：這是一個勾選框，使用 `ref` 屬性將 `checkboxRef` 參照指定給該元素，以便在需要時能夠存取 `checkbox` 元素。

`<button type="submit">Submit</button>`：這是一個提交按鈕，用於提交表單。當使用者點擊該按鈕時，將觸發表單的提交事件。

整個組件的運作方式如下：

初始化三個 `useRef` 參照，分別用於存取 `input`、`select` 和 `checkbox` 元素。

渲染一個包含文字輸入欄位、下拉選單、勾選框和提交按鈕的表單。

使用 `ref` 屬性將參照指定給相應的元素，以便在需要時能夠存取它們的值。

當使用者提交表單時，觸發 `handleSubmit` 函式，從參照中取得對應元素的值並輸出到控制台。

```
import { useRef } from "react";

export default function UncontrolledForm() {
  const selectRef = useRef(null);
  const checkboxRef = useRef(null);
  const inputRef = useRef(null);

  function handleSubmit(event) {
    event.preventDefault();
    console.log("Input value:", inputRef.current.value);
    console.log("Select value:", selectRef.current.value);
    console.log("Checkbox value:", checkboxRef.current.checked);
  }

  return (
    <form onSubmit={handleSubmit}>
      <label>
        <p>Name:</p>
        <input ref={inputRef} type="text" />
      </label>
      <p/>
      <label>
        <p>Favorite color:</p>
        <select ref={selectRef}>
          <option value="red">Red</option>
          <option value="green">Green</option>
          <option value="blue">Blue</option>
        </select>
      </label>
      <p/>
    </form>
  );
}
```

```
    <label>
      Do you like React?
      <input type="checkbox" ref={checkboxRef} />
    </label>
  <p/>
  <button type="submit">Submit</button>
</form>
);
}
```